

Staticfy – A static analysis tool

A report chronicling the development of a python script used to automate the static analysis of a file, to help determine its nature and write the information to a file for further triage.

Adam Henderson

CMP320: Advanced Ethical Hacking

2025/2026

Note that Information contained in this document is for educational purposes.

Abstract

A python-based scripting tool was developed to automate the process of Static Analysis on potentially malicious files involving the extraction of identifying information about the file such as its strings and imported DLL's as well as identifying the likelihood of obfuscation via packing. Due to the wealth of information that can be turned up by Static Analysis and the number of different tools and methods that can be used highlights the importance of a single script that performs all of the sections of analysis and collates the findings in one place automatically.

The developed script focusses on querying the database of popular virus analysis site "VirusTotal" using API integration and parsing the response data to identify the likelihood of the file causing harm to a user's system. A multitude of functions were implemented to allow for further analysis of the file such as string extraction, DLL extraction, packing detection and uploading of files to VirusTotal for analysis. The tool was developed with efficiency and further analysis in mind and as such the results of the analysis were written to an output file. The tool was written with a command-line interface in mind to allow for ease of use.

The script was tested using several different files such as the eicar.com anti-virus test file, the setup file for popular social media platform Discord and user written benign files. The script was capable of correctly determining the nature of the provided files, extracting the imported DLL's and strings and detecting the likelihood of the file being packed. The script was also able to upload files to the VirusTotal site for analysis and writing all of the important information to a user defined output file.

Contents

1	Introduction	1
1.1	Background.....	1
1.2	Aim.....	2
2	Methodology.....	4
2.1	Overview of Methodology.....	4
2.2	Basic Static Analysis	5
2.2.1	Hashing Of Files	5
2.2.2	Getting VirusTotal Response	5
2.2.3	Uploading a file to VirusTotal.....	5
2.3	Advanced Static Analysis	5
2.3.1	String Extraction.....	5
2.3.2	Imported DLL Discovery	5
2.3.3	Packing Detection.....	6
2.4	Script Design	6
2.5	Testing	6
3	Program and Development.....	8
3.1	Overview of Program Development.....	8
3.2	File Hashing.....	9
3.3	Querying VirusTotal	9
3.4	Extracting Strings	10
3.5	Extracting Dll's	11
3.6	Packing Detection	11
3.6.1	Entropy Calculation	11
3.6.2	Section Name Comparison	11
3.7	Uploading Files	12
3.8	Writing to file.....	12
3.9	Command-line interface	12
3.10	Testing	13
4	Discussion.....	16
4.1	General Discussion	16
4.1.1	File Hashing	16

4.1.2	Querying VirusTotal.....	16
4.1.3	Extracting Strings.....	16
4.1.4	Extracting DLL's	17
4.1.5	Packing Detection.....	17
4.1.6	Uploading files.....	17
4.1.7	Writing to file	17
4.1.8	Command-line interface	17
4.1.9	Testing.....	18
4.2	Future Work.....	18
References		19
Appendices.....		21
Appendix A – Contents of output files for testing		21
4.2.1	Output.txt file contents, results of testing on eicar.com	21
4.2.2	Output1.txt contents, results of testing on DiscordSetup.exe	22
4.2.3	Output5.txt file contents, proof of successful upload to VirusTotal.....	28
4.2.4	Ouptut6.txt contents, results of testing the verbose flag.....	29
4.2.5	Output2.txt contents, results of testing the docker implementation of the script	47
Appendix B – Script Code		48
Appendix C – Contents of the dockerfile		56
Appendix D – README file contents.....		57

1 INTRODUCTION

1.1 BACKGROUND

Aycock, 2006 defines malicious software or malware as software whose intent is malicious, or whose effect is malicious (Aycock, J. 2006). Malware encompasses a wide variety of harmful programs, the kinds of which have only become more sophisticated since 2006 when Aycock wrote about it. Now, almost 20 years later as of October 2025, the world is more connected than ever with internet users estimated at an all-time high of 6 billion (Kemp, S. 2025). It's also estimated that 33,540,000,000 internet connected devices are active as shown in Figure 1.

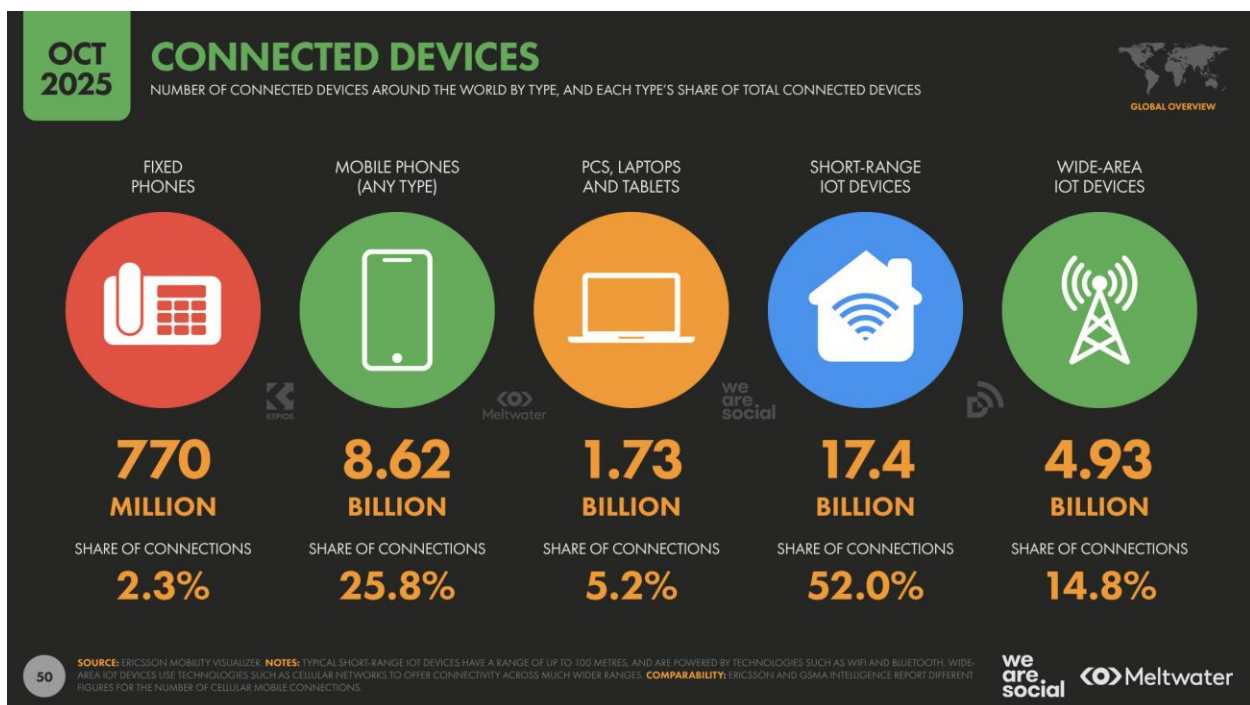


Figure 1 – Total number of internet connected devices categorised by type (Kemp, S. 2025)

The greater number of devices also means a greater number of malware targets, and the greater impact. Figure 2 displays the increasing number of malware attacks from 2008-2025.

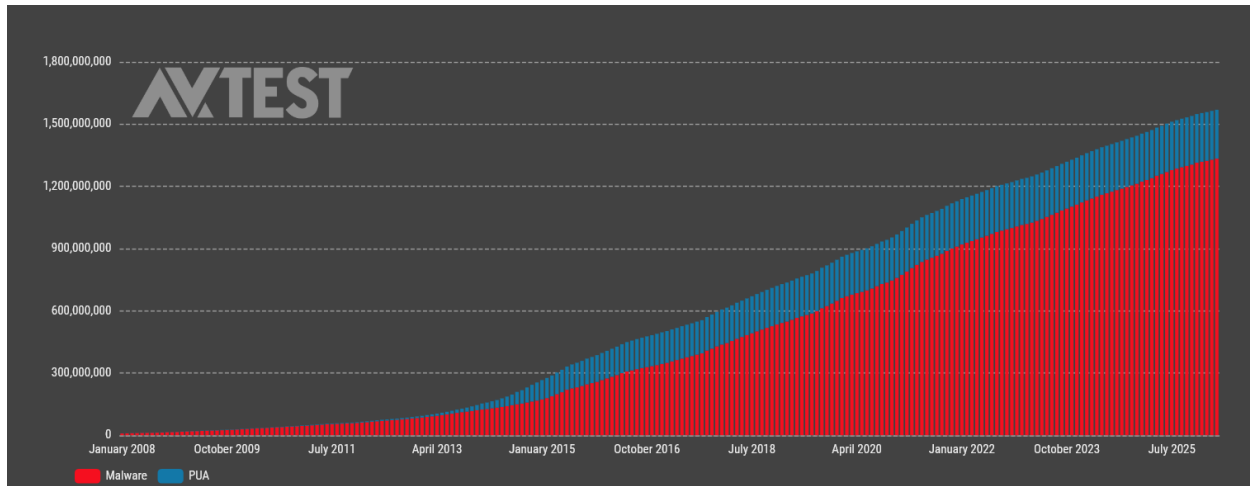


Figure 2 – Malware statistics in millions (AV-Test. 2026)

Globally, the average cost of a data breach in 2025 was \$4.44 Million (IBM. 2025) with all 3 of the most common vectors of attack being obtainable by malware among others it is clear that the harmful software is having a grave effect on business and consumers alike.

A method that can be used to protect against malware attacks is Static Analysis. Static Analysis is described by Gondotra et al in 2014 as analysing malicious software without executing it (Gondotra, E. Bansal, D. and Sofat, S., 2014). An issue with static analysis is that it is often time consuming and numerous features are extracted and analysed to determine the nature of the file. In Schultz et al's research numerous data mining techniques are used to analyse malware such as extracting strings, DLLs and byte sequences (Schulz, M.G. Eskin, E. Zadok, F. and Stolfo, S.J., 2001). An issue with this as discussed by Carvalho et al is that it is easy to obfuscate program byte sequence using various binary packing techniques such as UPX (Carvalho, M.M. Chan, K.P. and Hassen, M., 2017). This shows that it is extraordinarily difficult to perform efficient static analysis on files which may or may not be malware. The program which was developed carries the end goal of making the static analysis of files more automated and simpler, allowing for a quick determination of the files nature and dumping useful data into a file for further triaging allowing for both a surface level detection and further analysis to be performed at the users leisure and at a low cost.

1.2 AIM

This project aims to develop a python based automated Static Analysis tool for use on files to identify the likelihood of harm to a user's system. The specific aims required to achieve this are as follows:

- Using the VirusTotal database, determine the likelihood of a chosen file to be malicious to the user's system.
- Extract identifying information about the file such as strings and imported DLLs.

- Identify the likelihood of the given file being packed.
- Allow the user to upload unknown files to the VirusTotal database for analysis.
- Develop a command-line interface allowing for fluid user experience to analyse different types of files at varying degrees of intensity
- Evaluate the effectiveness of the script through anti-virus test files, benign files and user created files.
- Use docker to containerise the file allowing for increased portability and usability.

2 METHODOLOGY

2.1 OVERVIEW OF METHODOLOGY

The project used a structured and modular approach to Static analysis of malware, using the strings, VirusTotal results, DLL imports and whether or not the file was packed to identify the likelihood of the file being malicious and to provide a starting point for further malware analysis. An automation script using the Python programming language was created as a tool for this purpose. Python was chosen due to its simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development. The Python interpreter is easily extended with new functions and data types (Van Rossum, G. and Drake, F.L., 2003).

The general development approach used consisted of 7 modular sections. Initially, a function was created to generate the md5 and SHA256 hash value of a chosen file; these values were then printed to the console. Next, the hash values of the files were used to query the API of the site VirusTotal. VirusTotal. VirusTotal inspects items with over 70 antivirus scanners and URL/domain blocklisting services (VirusTotal., 2026). The response data was then parsed and used to determine the danger posed by the file. Safe means that no danger was detected, malicious implies that the file is likely malware, and unknown suggests that the file may be malicious; however further analysis is required. If the file was not present in the VirusTotal database, the user would be able to upload the file to the site for it to be analysed.

Further development of the program focused on the more technical user, gearing itself towards malware analysis. Initially, a string extraction function was implemented to help analysts decipher what a piece of malwares end goal may be. Next, a function to discover the imported DLL's (Dynamic Link Library) was implemented. A DLL is a library that contains code and data that can be used by more than one program at the same time (Microsoft., 2026). DLL's often give an insight into the operations of a file and can give analysts an idea of what a piece of malwares functionality may be. Finally, a pair of functions are used to check if a file has been packed. Packers are software programs that compress and encrypt other executable files in a disk and restore the original executable images when the packed files are loaded into memories (Yan, W. Zhang, Z. and Ansari, N., 2008). To test if the file has been packed an entropy calculation was used and a number of suspicious file extensions are compared to section names to guess if the file is packed or not.

To support usability the tool was developed as a command-line interface tool to allow for customization of commands. The effectiveness of the tool was assessed using anti-malware test files such as eicar.co, benign files such as DiscordSetup.exe and random user made text files to ensure that upload was working correctly. This ensures that the tool covers all bases and was effective in all areas of its design.

2.2 BASIC STATIC ANALYSIS

The program aims to simplify the process of Static analysis of malware. To perform this the programs functionality is split into two sections. Basic Static Analysis and Advanced Static Analysis. Basic Static Analysis is used to give an indication of the files likelihood to cause harm and allows non-technical users peace of mind.

2.2.1 Hashing Of Files

Initially, the project focused on uploading a hash value of a chosen file by the user to confirm the safety of the file. To perform this the tester got the md5 and SHA256 hash of the file using the python hashlib library. Files were processed in 64 Byte chunks to ensure that larger files were processed properly without putting too much strain onto the CPU.

2.2.2 Getting VirusTotal Response

The next stage of the project involved using the VirusTotal API v3 to get the data surrounding the chosen file. This was performed using the requests library in python, Requests allows you to send HTTP/1.1 requests extremely easily (PyPI., 2026). The json data retrieved from the request was then parsed using the json library to find the most recent VirusTotal analysis' results. This data is then categorised into python variables and used to determine the likelihood of the safety of the file.

2.2.3 Uploading a file to VirusTotal

The final stage of simple Static malware analysis was to allow the user to upload the chosen file to VirusTotal using the API v3 upload endpoint. Again, the requests library is used send a POST request to upload the file, once it has been analysed the script can be used again to identify if the file is malicious or not.

2.3 ADVANCED STATIC ANALYSIS

The second section of the program focuses on supplying a "jumping off point" for more technical users to explore the capabilities of a file, to understand the workings of it and to provide extra information to help aid in the investigation of suspicious files without having to use a multitude of programs.

2.3.1 String Extraction

The first stage of advanced static analysis involves string extraction. By reading through each byte of data in the chosen file and confirming the string is a printable character and not a null character it is extracted into a list which can be used to further analyse the file and give an idea on what its capabilities are.

2.3.2 Imported DLL Discovery

Next, the discovery and storage of DLL's is performed using the pe library. pe is a library for parsing expressions, including parsing expression grammars (PEGs). It aims to join the expressive power of parsing expressions with the familiarity of regular expressions (PyPI., 2026). Returning the DLL's allows analysts a more in depth look at the functionality of the file and what it may do when ran.

2.3.3 Packing Detection

Finally, packing detection was implemented. The math library was used to perform entropy calculations according to the Shannon Entropy formula. If the entropy is high, packing is likely. Another method used was to compare the section names within the file with the names of sections that are commonly packed. This shows analysts whether there is more to the file than meets the eye upon initial testing.

2.4 SCRIPT DESIGN

The methodology used for the scripts design was a structured and modular approach mapped to the MoSCoW prioritisation. The MoSCoW prioritisation framework is an acronym that stands for Must have, Should have, Could have, and Won't have. This framework provides a structured approach to categorising tasks or requirements based on their importance (BiteSize Learning., n.d.). The tool uses this approach for efficiency of processing, readability, maintainability and allows for additional features to be added easily. Each function performs an isolated step vital for the proper analysis of a file.

The overall workflow begins with the hashing of a file; these hashes are printed to the console and then used to query the VirusTotal API. The response from the API is parsed to isolate and extract the most recent analysis stats. This can be enriched by the user via the command line interface so that the entire response is extracted instead by using the verbose flag. Once the analysis results are identified, a rule-based system is used to determine the likelihood of the file being malware.

After basic malware analysis has been achieved, the tool begins further analysis by extracting all of the string present in the chosen file. The user can set a minimum length for this using the minimum length flag, the script runs with a default of 4-character minimum length. Next, the discovery and logging of imported DLLs is performed. Finally, the script calculates the entropy of each section of the file and checks if the name of the section matches common packing or obfuscated names to advise the user if the file is packed or not.

Following the completion of the analysis the tool outputs the analysis statistics parsed from VirusTotal, the likelihood of the file being harmful based on rule-based detection, the strings within the file, the imported DLLs and the likelihood of packing by writing them to a user defined file for further analysis or for recording and logging.

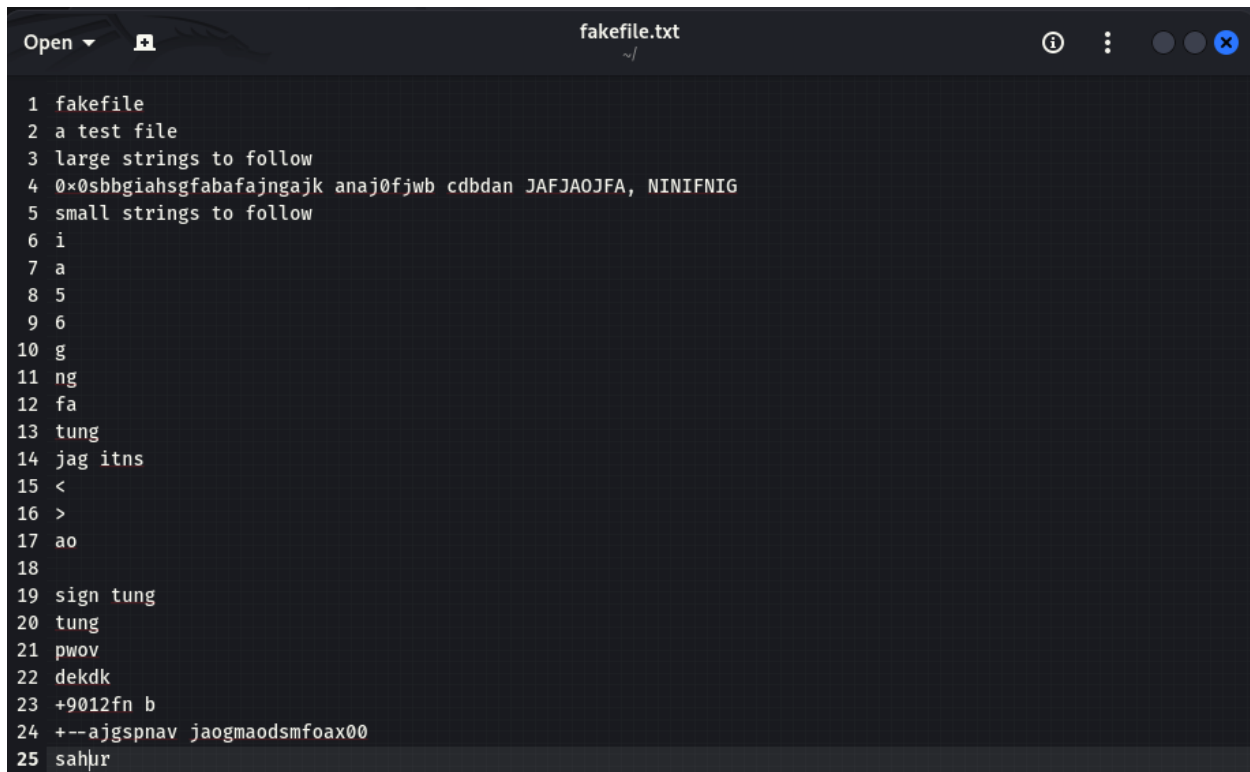
To ensure that the tool is as useful as possible, an upload feature was implemented to allow the user to upload a file to VirusTotal so that it can be analysed and the script can be run again to confirm its safety.

2.5 TESTING

The capabilities of the script were tested using four different kinds of file. One benign packed file being the DiscordSetup.exe, One benign unpacked file written by the user to test the upload functionality and the anti-malware testfile eicar.co.

These files were selected to ensure that the tool responds as expected to benign files and files that should be picked up by antivirus, The use of .exe files allows for proper testing of the DLL logging

functionality and the use of an installer allows testing of the packing functionality. String extraction will be covered by testing the script on all files and ensuring strings are picked up correctly across all files. The user created file used for testing can be viewed in Figure 3 below.



```
1 fakefile
2 a test file
3 large strings to follow
4 0x0sbbgiahsgfabafajngajk anaj0fjwb cdbdan JAFJAOJFA, NINIFNIG
5 small strings to follow
6 i
7 a
8 5
9 6
10 g
11 ng
12 fa
13 tung
14 jag itns
15 <
16 >
17 ao
18
19 sign tung
20 tung
21 pwov
22 dekdK
23 +9012fn b
24 +--ajgspnav jaogmaodsmfoax00
25 sahur
```

Figure 3 – The user created fakefile.txt for testing

To ensure the scripts robustness, deliberately odd or illogical inputs were provided by the user when initiating the script. This was done to ensure that the script can deal with strange file paths, incorrect command usage or incorrect arguments.

3 PROGRAM AND DEVELOPMENT

3.1 OVERVIEW OF PROGRAM DEVELOPMENT

The program was developed using a modular approach. Initially, the workflow was defined to allow for the program to be worked on sequentially. The initial workflow was:

- Hash files
- Query VirusTotal
- Extract Strings
- Extract imported DLLs
- Detect Packing
 - Entropy Calculation
- Upload file
- Write to file
- Implement Command-line interface

Each section was developed individually to allow for simple debugging and a focus on understanding what each section does, allowing for better documentation. A large focus was also placed on robustness allowing for each section to be written to handle errors gracefully. Each section being programmed individually allowed for sequential testing of each component of the program. The end goal of the program was to create a command-line interface tool which automates the process of static analysis of malware, dumping strings, DLLs, checking for packing and giving a judgement on the threat level of a given file then storing the information in an output file for further triage.

To begin the program needed to be able to compute the MD5 and SHA256 hashes of a given file, these would then be passed to the function that queries VirusTotal. In conjunction with the VirusTotal API v3 the SHA256 hash value was used to get a report on the file from the VirusTotal database. The data returned from this was parsed and used to determine the nature of the file based on an arbitrary number of detections per total vendors who processed the file.

After the data could reliably be collected and parsed, implementation of more advanced malware analysis methods was implemented. This began with the extraction of strings in the file; each ASCII printable string was identified and if it was greater than the user specified minimum string length it would be stored in a list. Following this, imported DLLs would be extracted from a PE file and stored. Next, detection of the file being packed was implemented. There were 2 methods to discovering this, in each section of a PE file the entropy would be calculated, if the resulting number was greater than 7 the file was likely to be packed. To compliment this section names were checked against common section names for packed files.

Having completed the Static Analysis, a quality-of-life component was implemented which allows the user to upload a file to VirusTotal if it has not been scanned, the script can then be run again as normal to triage the file.

Next, the results of the analysis were written to a user specified output file to allow for further analysis.

Finally, to make the tool as user friendly as possible; a command-line interface was implemented using the argparse library. This allows the user to specify the minimum length of strings to be extracted, the verbosity of the virus total results and the file upload component to be used allowing for more fluid analysis of files.

3.2 FILE HASHING

The implementation of the file hashing function allowed for computation of the SHA256 and MD5 hash of a given file. Initially a buffer size of 64 bytes was set to define how much of a file would be processed at one time. This was implemented to allow for improved efficiency when processing larger files. Then the MD5 and SHA256 hashes were initialised using the hashlib library.

A file was opened in binary mode for improved portability and so that data would be returned as bytes objects, allowing for easier processing of the data within the file. A loop was created to process the file in chunks and only break at the end of the file. Upon each iteration of the loop, the MD5 and SHA256 hash values are updated. Once the end of the file is reached the hexadecimal values of the hashes are printed and returned.

To maintain robustness the opening of the file used the with open method contained within a try except statement. This ensures the file will be properly closed when the processing of data has been complete and that any errors are handled and printed on screen before exiting the script.

3.3 QUERYING VIRUSTOTAL

The implementation of querying VirusTotal was the backbone of the program, getting results from the VirusTotal database provides a baseline for the nature of the chosen file, and can give powerful insights into what the file can do. To implement this, the hash values of the file must be generated prior.

To set up the querying of VirusTotal the URL and headers were defined in line with the VirusTotal API documentation, the final URL section was appended with the generated SHA256 hash. SHA256 was used for this as it is the most used hash type on VirusTotal and therefore will return a result if one exists on the database whereas MD5 may not be stored. The headers included returning json type data, as it is simple to parse in python and the API key needed for authentication. In order to keep the private API key from being leaked, an environment variable was used.

To query the database the requests library was used, and the response was returned as a tuple containing the API response and API object to allow for parsing.

In the main function, the nested json data is parsed by extracting the last analysis stats object of the json data and defining integer variables to store the number of malicious, suspicious, undetected and harmless results found by the security vendors. The total number of security vendors who processed the file is calculated. This data is used to determine the nature of file, the rules implemented state that if the number of malicious detections is greater than half of the total number of security vendors who processed the file it is malicious, if no security vendors detected the file as malicious or suspicious it is

safe and any other outcome means it is unknown and further analysis should be performed. These results are printed to the screen for the user.

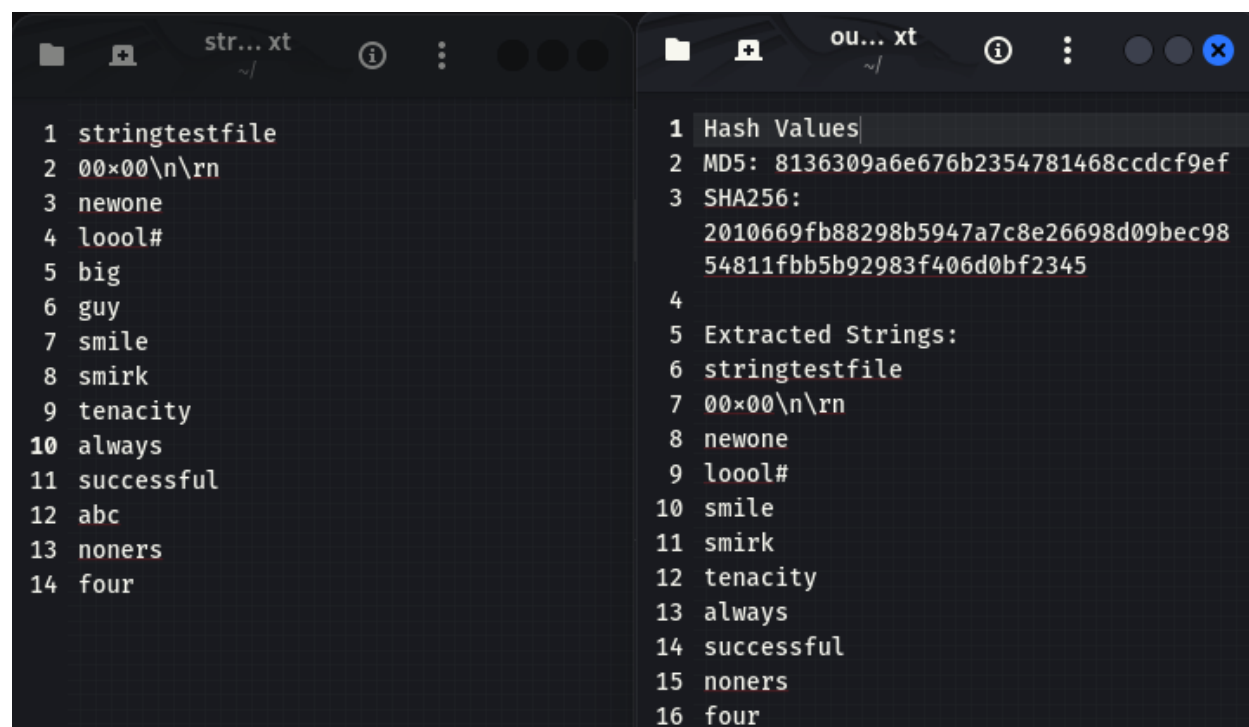
To ensure robustness a try except statement is used to handle errors gracefully such as lack of authentication or if the file wasn't in the VirusTotal database and inform the user of the issue with querying the API so that they may try again.

3.4 EXTRACTING STRINGS

After the script has queried VirusTotal, string extraction takes place. The implementation of string extraction involves creating a list of strings to be stored. Then the file is opened in binary to read through, this is processed using the "with open" method and wrapped in a try except statement to deal with file read or opening errors.

The program then parses through the file byte by byte and adding characters to the "current" variable to store them. In each iteration, the characters are confirmed to be printable by ASCII and not whitespace or null characters. Then the length is checked against the minimum length specified by the user or the default of 4. The string list is then appended, and current is reset to allow for new strings to be added. The list of strings is returned to the main to allow for triage after they are dumped to the file.

To ensure this works a test file called "stringtestfile.txt" was created with numerous strings, the output compares the strings found to ensure they have been correctly parsed, a minimum length of 4 is used. This is shown in Figure 4.



```
1 stringtestfile
2 00x00\n\rn
3 newone
4 loool#
5 big
6 guy
7 smile
8 smirk
9 tenacity
10 always
11 successful
12 abc
13 noners
14 four

1 Hash Values
2 MD5: 8136309a6e676b2354781468ccdcf9ef
3 SHA256:
4 2010669fb88298b5947a7c8e26698d09bec98
5 54811fbb5b92983f406d0bf2345
6
7 Extracted Strings:
8 stringtestfile
9 00x00\n\rn
10 newone
11 loool#
12 smile
13 smirk
14 tenacity
15 always
16 successful
17 noners
18 four
```

Figure 4 – comparison of "stringtestfile.txt" and output of script's strings extraction

3.5 EXTRACTING DLL'S

The implementation of DLL extraction involves the use of the pefile library, this is because DLLs are present only in portable executables. Portable Executables (PE's) describes the structure of executable (image) files and object files under the Windows family of operating systems (Microsoft., 2025). To store and return these imports, a list is used, then the file is scoured for the "DIRECTORY_ENTRY_IMPORT" attribute which is where DLLs are loaded. Once it is confirmed that the file is a PE and has imported DLL's a loop begins for every import within the "DIRECTORY_ENTRY_IMPORT" attribute. Every import is decoded using the utf-8 character set and appended onto the list.

To ensure proper error handling, a try except statement is used to inform the user of any PE Parsing errors, the program continues running as not all files are PE's.

The list of DLLs is returned to be further explored after being written to the output file

3.6 PACKING DETECTION

Packing detection estimation was implemented in this script using the pefile library, initially a Boolean variable called packed was set to false, and a list of suspicious names of sections were defined. It was important to note that packing detection is very difficult and even harder to get right every time, so the script was modified to ensure that the output was not too decisive.

The script iterates through each section of the file if it is a PE file, in each iteration it decodes the names of the sections and strips them of null bytes and stores them in a variable called "name". It also gets the data of the section and stores it in a variable called "data".

3.6.1 Entropy Calculation

The first method used to determine if packing was used is the use of an entropy calculation. This occurred in a separate function and made use of the math library. For this Shannon's Formula was used. Entropy refers to a measurement of vagueness and randomness in a system (Karaca, Y. and Moonis, M., 2022). Shannon's Entropy formula is shown below:

Shannon's entropy equation:

$$H(X) = - \sum_{i=0}^{N-1} p_i \log_2 p_i$$

Figure 5 – Shannon's entropy equation image (Kaplan, I. 2002)

Using this formula the value of entropy is calculated, if entropy is larger than 7 the file is very likely to be packed. The entropy value is returned to the main packing detection function where it is checked and if it is greater than 7.0 packed is set to true.

3.6.2 Section Name Comparison

The second method used to determine if the file is packed is section name comparison. The decoded names of the sections of the PE file are compared to a list of section names commonly found in packed

files. The script iterates through the list of suspicious names and checks if the lowercase version of the commonly found packed section names were in the name of the current PE file section. If a match is found, then packed is set to true.

The value of packed is returned to main to be checked and the results of the packing detection are written to the file.

A try except statement is used to handle errors with the parsing of the PE file.

3.7 UPLOADING FILES

The implementation of the uploading files function requires the use of the VirusTotal API. It also requires the guessing of a file type as the type needs to be used to properly setup the file so it can be uploaded to the VirusTotal database. A try except statement is used to catch any errors whilst uploading the file. The function is written according to the VirusTotal v3 API documentation.

3.8 WRITING TO FILE

Writing to file is the final step of the static analysis section of the program, the file write function is used to allow further analysis of results and to collate the results of the static analysis script. This is performed by using the “with open” method and a try except statement to handle errors. Within this, all the information is written to the file using the “f.write” function.

3.9 COMMAND-LINE INTERFACE

The tool was designed with a command-line interface function in mind, allowing end users to execute the script with a number of different options, most importantly the ability to freely edit the file path of the target file and the output file for triage and results. Other flags are implemented for more verbose VirusTotal results and, minimum length of strings extracted and to upload a file. This allows the tool to be easily integrated into current workflows and ongoing projects. It also means that the tool is more lightweight and portable as it doesn't require a graphical interface.

The command line interface was implemented using the argparse library. This allows for user defined arguments to be parsed by the script and executed as the script is run. This script is made up of 2 different kinds of arguments, positional and optional.

Positional arguments are required in every operation, for the malware triage a file path to the target file to be investigated is always required, an output file is also always required.

The optional arguments consist of an upload argument which allows the user to use the upload file function and have a file of their choice analysed by VirusTotal, another optional argument is the verbose argument. Verbose means that after parsing the json data returned by querying the VirusTotal API includes the entire response along with the filtered response of the most recent analysis. Finally, the minimum length argument is used to set the minimum length of extracted strings, allowing the user to target specific sizes of strings only or to cover a wider range of strings.

To provide users with guidance the help option was also implemented. It displays guidance on each optional flag and the way the tool should be used as standard allowing for greater usability.

The standard method of executing the tool within the command line is:

```
Python Staticfy.py [target file path] [outputfile.txt]
```

This command invokes the python script and specifies the target file and the output text file that the results should be written to.

3.10 TESTING

To test the scripts functionality several different files were used, initially the eicar.com anti-virus test file was used to confirm that the script can detect malicious files without the possibility of harming the system. The results of this test can be seen in Figure 6. The full results can be seen in Appendix A.

```
abertay@abertay-kali:~$ python Staticfy.py /home/abertay/Downloads/eicar.com output.txt
MD5: 44d88612fea8a8f36de82e1278abb02f
SHA256: 275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f
PE Parsing error: 'DOS Header magic not found.'
Packing check error: 'DOS Header magic not found.'
Likely [*]Malicious[*]
This file was identified as malicious by 66 of 68 Security Vendors
```

Figure 6 – The results of running the script on the eicar.com antivirus file

Next, the DiscordSetup.exe file was tested. This was used to confirm the system could identify a benign file as well as detect packing, extract strings of a minimum length specified by the user and identify imported DLLs. The command line response is shown in Figure 7 and the full of the results can be found in Appendix A.

```
abertay@abertay-kali:~$ python Staticfy.py /home/abertay/Downloads/DiscordSetup.exe output1.txt -m 25
MD5: 48e306701bc6540d3babf868a317fa0e
SHA256: 3f0306caceaa9594c608b39dbec8ceaba4422abc51052ee21deda7280eee9173
Likely [v]SAFE[v]
0 Security Vendors identified this file as malicious or suspicious
```

Figure 7 – The results of running the script on the DiscordSetup.exe file of a minimum string size of 25

As seen in Appendix A, the 2 imported DLLs found by the script were KERNEL32.dll and COMCTL32.dll to confirm the accuracy of this the malware analysis tool Detect It Easy (die) was used to view the imports. As shown in Figure 8, the scripts findings match with the findings from die confirming that it works as expected.

	ginalFirstTh	neDateStan	orwarderCha	Name	FirstThunk	Hash	
0	0002f2f0	00000000	00000000	0002f838	00024008	176ce022	KERNEL32.dll
1	0002f2e8	00000000	00000000	0002f85e	00024000	f94bcf1c	COMCTL32.dll

Figure 8 – The results of die's analysis on the DiscordSetup.exe import checks

To ensure that the upload component was functioning correctly a user created file was used, this can be seen in Figure 3 in section 2.5. The tester attempted to use the upload flag have the chosen file analysed by VirusTotal, the result can be seen in Figure 9.

```
abertay@abertay-kali:~$ python Staticfy.py -u fakefile.txt output5.txt
MD5: d48faf20a0fabd520c9bcadd0a00b563
SHA256: 02cdcc5c5a84b3b52f520cff63ccda057fc3db36c8f2de81c22c3d7ef0c273cc
File upload successful
```

Figure 9 – Results of uploading a chosen file to VirusTotal

To ensure this worked, the standard execution of the tool was used to query VirusTotal for the recently uploaded file. The results can be seen in Figure 10.

```
abertay@abertay-kali:~$ python Staticfy.py fakefile.txt output5.txt
MD5: d48faf20a0fabd520c9bcadd0a00b563
SHA256: 02cdcc5c5a84b3b52f520cff63ccda057fc3db36c8f2de81c22c3d7ef0c273cc
PE Parsing error: 'DOS Header magic not found.'
Packing check error: 'DOS Header magic not found.'
Likely [v]SAFE[v]
0 Security Vendors identified this file as malicious or suspicious
```

Figure 10 – The recently uploaded file being checked using the script

The full analysis results can be viewed in Appendix A

Further robustness testing was conducted such as setting the minimum length to a value less than 1 which yielded the result shown in figure 11.

```
abertay@abertay-kali:~$ python Staticfy.py /home/abertay/Downloads/eicar_com2.zip outpu3t.txt -m 0
Minimum length must be 1 or greater
```

Figure 11– results of attempting to set the minimum length too low.

Finally, the verbose flag was tested to ensure that all of the VirusTotal data was dumped. This can be viewed in Appendix A. Figure 12 shows the command line response of the tool.

```
abertay@abertay-kali:~$ python Staticfy.py fakefile.txt output6.txt --verbose
MD5: d48faf20a0fabd520c9bcadd0a00b563
SHA256: 02cdcc5c5a84b3b52f520cff63ccda057fc3db36c8f2de81c22c3d7ef0c273cc
PE Parsing error: 'DOS Header magic not found.'
Packing check error: 'DOS Header magic not found.'
Likely [v]SAFE[v]
0 Security Vendors identified this file as malicious or suspicious
```

Figure 12 – results of testing verbose flag

To ensure ease of use and portability, A docker container was used. To test and insure this worked the eicar.com testfile was used within the shell within the container. The results can be seen in figure 13. The full results can be found in Appendix A.

```
root@aa36fa27a4f3:/app# python Staticfy.py eicar.com output2.txt
MD5: 44d88612fea8a8f36de82e1278abb02f
SHA256: 275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f
PE Parsing error: 'DOS Header magic not found.'
Packing check error: 'DOS Header magic not found.'
Likely  [*]Malicious[*]
This file was identified as malicious by 66 of 68 Security Vendors
```

Figure 13 – results of running the script via docker

The dockerfile used can be found in Appendix C.

Additionally, to inform the user on how to use the script and to containerise it inside of docker as well as a brief description of the script. A README file was included in the project files. This can be viewed in Appendix d.

4 DISCUSSION

4.1 GENERAL DISCUSSION

The development of the Staticfy.py script was a major success as every component intended to be implemented was successfully written and tested extensively. The choice of using a modular and sequential approach to testing proved to be an excellent decision as it allowed the developer to build upon strong foundations and rational workflows. It also enhanced testing and debugging and allowed for robust, well documented code to be formed at every stage. The program successfully determines the likelihood of files harming the user's system and writes important information to a user specified output file. The specific goals were all met with high degrees of success. The determination of the likelihood of a file being malicious to the user's system was correct in every instance, the extraction of identifying information about the file was successful and retrieved full and correct information every time and laid the information out in a file in an easy, readable manner. The identification of the likelihood of the file being packed was correct in all testing and the ability to upload an unknown file to VirusTotal's database was successful. Furthermore, the program was successfully developed into a command-line interface tool with several options that allow for a fluid and simple end user experience. The testing performed on the tool was thorough and successful and docker was successfully utilised to allow ease of use and portability to the script. All the sections of the script which work with file handling use with open statements to properly open, read, write and close the file, they also use try except statements for graceful error handling.

4.1.1 File Hashing

The file hashing phase of development was implemented efficiently and was found to be successful throughout testing. In particular, the use of a buffer size to read the file in small chunks for efficiency was a powerful adjustment that allowed for larger files to be processed with ease. The use of multiple hashing algorithms allowed for more user confidence. Additionally, the use of the hashlib library increased efficiency as its built-in functions for computing hashes saved time and user code.

4.1.2 Querying VirusTotal

The querying VirusTotal phase of development was implemented using an API for smooth integration into the script. Using the provided documentation on the VirusTotal website the choice to query the site using the SHA256 hash was made as it is the most found hash type on VirusTotal, this means less errors and less uploads are required from the user. The use of the requests was pivotal to this stage of development as it allowed for easy crafting of a HTTP GET request to receive the information in json format. This json format was then parsed in the main function and stored in variables which were used later in the script to determine the likelihood of the file causing harm to the user's system, completing one of the specific aims.

4.1.3 Extracting Strings

The string extraction phase of development was implemented by parsing the strings found in the file by reading the data in bytes. This was done for efficiency and to allow the user to implement a minimum length of string to extract. These strings were parsed and appended to a list which was returned and

written to a file for further analysis. The developer would have found this section to be more useful in further analysis if a function for filtering strings was implemented, allowing the user analysing them to find suspicious strings more easily.

4.1.4 Extracting DLL's

The extraction of DLL's was performed using the pefile library to decode the names of imported DLL's and stored them in a list to be analysed. This section of code implemented a try except statement which was used for the handling of errors and – more importantly – to allow non-PE files to be processed as normal without imported DLL analysis. Initially the developer intended to extract the names of functions contained within the imported DLL's which would have been a major upgrade to the script. This completes the specific aim of extracting identifying information from the file.

4.1.5 Packing Detection

The packing detection was performed using multiple methods in an attempt to get a more accurate estimate of whether or not packing was used. These 2 methods were entropy calculation and suspicious name comparison. Entropy calculation uses Shannon's formula to calculate the section of the files, found by using the pefile library, entropy. If the value calculated is greater than 7 the file is likely to be packed. The method of suspicious name comparison uses the decoded name of each section of the file compared to a predefined list of section names commonly found in packed files. This section of code also implemented the try except statement to handle errors and allow for the processing of non-PE files. It is important to note that whilst entropy calculations and name comparison are good methods to determine if a file is packed, there is a high chance of false negatives and as such all information related to packing should only state that it may be in place as a concrete answer is difficult to ascertain. This meets the specific aim of identifying the likelihood of the file being packed.

4.1.6 Uploading files

The uploading files section was handled using API integration with VirusTotal. Again, the code was implemented following the official VirusTotal API documentation to ensure proper implementation. The requests library was used this time to craft a HTTP POST request. A point of interest is that the upload of files larger than 32 megabytes in size to the VirusTotal database requires a different API endpoint than the one used in this script, this should be implemented when possible as an optional flag. This completed the specific aim of being able to upload files to VirusTotal for analysis.

4.1.7 Writing to file

The writing to files section was largely used to collate the information in one place for easier triage, analysis and documentation. It uses a with open statement wrapped in a try except statement for proper file and error handling.

4.1.8 Command-line interface

The implementation of a command line interface was handled largely in the main function using the argparse library. Argparse is a library that lets the program define what arguments it requires, and argparse will figure out how to parse those out of sys.argv. The argparse module also automatically generates help and usage messages (Python. 2026). These arguments provide the user with a fluid and adaptable command-line interface that lets them use different flags for different functions and allows them to easily choose the files to analyse. This also completed the specific aim of implementing a command-line interface

4.1.9 Testing

The testing phase of the program development involved running the script against eicar.com an anti-virus testfile to ensure the malicious output was displayed without cause any real danger to the system. The script was also run against the DiscordSetup.exe to ensure DLL extraction, minimum length string extraction and packing detection worked, this had the added benefit of ensuring that benign files are identified as safe. Finally, a user submitted file was uploaded to test that it would be analysed by VirusTotal. This section also tested for handling user input error by ensuring the script didn't break upon receiving inputs such as a minimum length of 0 or less, the program had no issues handling this. This section also involved the user containerising the script into a docker which allowed for portability and ease of use a README file was included within the scripts directory to inform the user of the command usage, how to build and run the script in docker and a brief description of the script. This section completes the final pair of specific aims of performing proper testing on the script and containerising the script.

4.2 FUTURE WORK

A more sophisticated script may wish to implement multiple upgrades to some of the scripts functions. One area that could benefit from refinement is the DLL logging function. As of now the script extracts the names of the imported DLLs and writes them to the files, however each imported DLL may contain functions that are also imported. These functions can contain vital information pertaining to the purpose of the file. A more advanced script may wish to log these functions and write them to the file along with the DLLs they are imported under.

Another area that could benefit from refinement is the upload function, after uploading the file to VirusTotal the script exits and to get analysis results the script must be rerun. A quality-of-life improvement would be automatically running the analysis after uploading the file, removing the need to rerun the script. Another area of refinement would involve using the "Get a URL for uploading large files" endpoint. Whilst uploading files smaller than 32MB can be performed with the current scripting setup, for larger files you need to obtain a special upload URL first and then send the POST request to the upload URL instead of sending it to /files (VirusTotal. 2026). This would allow users to analyse more files as there is no upload limit.

Another helpful improvement could be made to the string extraction function, allowing the user to filter for IP addresses, URLs or specific requests using regex would allow for more confidence in analysis and refine the string extraction from a dump of strings to more identifiable information.

A final area of immediate improvement is the packer detection function. Despite correctly identifying the if the file is likely packed or obfuscated, the script doesn't give the user any indication what packer has been used or how to unpack the file. A powerful refinement would be identifying the type of packer used and writing the results to a file for the user to make use of.

REFERENCES

AV-Test. 2026. *Total amount of malware and PUA*. [online image]. Available From: <https://portal.av-atlas.org/malware> [Accessed 17 April 2026]

Aycock, J., 2006. *Computer Viruses and Malware*. New York: Springer New York, p.2.

BiteSize Learning. N.d. *The MoSCoW Prioritisation model, explained*. [online] Available at: <https://www.bitesizelearning.co.uk/resources/moscow-prioritisation-model> [Accessed 22 April 2026]

Carvalho, M.M. Chan, K.P. and Hassen, M. (2017) 'Malware Classification Using Static Analysis Based Features', *Proceedings of the 2017 IEEE Symposium Series on Computational Intelligence (SSCI)*. Honolulu, HI, USA, November 27-01 December, IEEE. pp. 1-7. Available at: <https://doi.org/10.1109/SSCI.2017.8285426>

Gandotra, E. Bansal, D. and Sofat, S., 2014. Malware Analysis and Classification: A Survey. *Journal of Information Security*, [online] Volume 5(2), pp. 56-64. Available at: https://www.researchgate.net/profile/Ekta_Gandotra/publication/276495476_Malware_Analysis_and_Classification_A_Survey/links/58e464c9aca272d62977a2b6/Malware-Analysis-and-Classification-A-Survey.pdf [Accessed 17 April 2026]

IBM. 2025. *Cost of a Data Breach Report 2025*. [online] Available at: https://www.bakerdonelson.com/webfiles/Publications/20250822_Cost-of-a-Data-Breach-Report-2025.pdf [Accessed 17 April 2026]

Kaplan, I. 2002. *Shannon Entropy*. [online image]. Available from: http://bearcave.com/misl/misl_tech/wavelets/compression/shannon.html [Accessed 24 April 2026]

Karaca, Y. and Moonis, M., 2022. *Multi-Chaos, Fractal and Multi-Fractional Artificial Intelligence of Different Complex Systems*. Elsevier. P.236.

Kemp, S. 2025. *Digital 2026: Global Overview Report*. [blog]. Available From: https://datareportal.com/reports/digital-2026-global-overview-report?utm_source=Global_Digital_Reports&utm_medium=Article&utm_campaign=Digital_2026 [Accessed 17 April 2026]

Kemp, S. 2025. *Digital 2026: Global Overview Report*. [online image]. Available From: https://datareportal.com/reports/digital-2026-global-overview-report?utm_source=Global_Digital_Reports&utm_medium=Article&utm_campaign=Digital_2026 [Accessed 17 April 2026]

Microsoft. 2025. *PE Format*. [online] Available at: <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format> [Accessed 24 April 2026]

Microsoft. 2026. *What is a DLL*. [online] Available at: <https://learn.microsoft.com/en-us/troubleshoot/windows-client/setup-upgrade-and-drivers/dynamic-link-library> [Accessed 20 April 2026]

PyPI. 2025. *pe 0.6.0*. [online] Available at: <https://pypi.org/project/pe/#description> [Accessed 21 April 2026]

PyPI. 2026. *Requests 2.33.1*. [online] Available at: <https://pypi.org/project/requests/#description> [Accessed 21 April 2026]

Python. 2026. *argparse — Parser for command-line options, arguments and subcommands*. [online]. Available at: <https://docs.python.org/3/library/argparse.html> [Accessed 28 April 2026]

Schultz, M.G. Eskin, E. Zadok, F. and Stolfo, S.J. (2001) 'Data mining methods for detection of new malicious executables', *Proceedings of the 2001 IEEE Symposium on Security and Privacy. S&P 2001*, Oakland, CA, USA, 14-16 May. IEEE. pp. 38-49. Available at: <https://doi.org/10.1109/SECPRI.2001.924286>

Van Rossum, G. and Drake, F.L., 2003. *An Introduction to Python*. Bristol: Network Theory Ltd, p.3.

VirusTotal. 2026. *How it works*. [online] Available at: <https://docs.virustotal.com/docs/how-it-works> [Accessed 20 April 2026]

VirusTotal. 2026. *Get a URL for uploading large files*. [online]. Available at: <https://docs.virustotal.com/reference/files-upload-url> [Accessed 28 April 2026]

Yan, W. Zhang, Z. and Ansari, N., 2008. Revealing Packed Malware. *Journal of IEEE Security & Privacy*, [online] Volume 6(5), pp. 65-69. Available At: <https://doi.org/10.1109/MSP.2008.126> [Accessed 20 April 2026]

APPENDICES

APPENDIX A – CONTENTS OF OUTPUT FILES FOR TESTING

4.2.1 Output.txt file contents, results of testing on eicar.com

```
1 Hash Values
2 MD5: 44d88612fea8a8f36de82e1278abb02f
3 SHA256: 275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabbf651fd0f
4
5 Extracted Strings:
6 X50!P%QAP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
7
8 VirusTotal Data:
9 Malicious: 66
10 Suspicious: 0
11 Undetected: 2
12 Harmless: 0
13 Total Vendors: 68
14
15 Imported DLLs
16 No DLLs found or not a PE file
17 Packing Analysis:
18 Not likely packed
```

4.2.2 Output1.txt contents, results of testing on DiscordSetup.exe

```

1 Hash Values
2 MD5: 48e306701bc6540d3babf868a317fa0e
3 SHA256: 3f0306caceaa9594c608b39dbee8ceaba4422abc51052ee21deda7280eee9173
4
5 Extracted Strings:
6 !This program cannot be run in DOS mode.
7 `vector deleting destructor'
8 `default constructor closure'
9 `scalar deleting destructor'
10 `vector constructor iterator'
11 `vector destructor iterator'
12 `vector vbase constructor iterator'
13 `virtual displacement map'
14 `eh vector constructor iterator'
15 `eh vector destructor iterator'
16 `eh vector vbase constructor iterator'
17 `copy constructor closure'
18 `local vftable constructor closure'
19 `placement delete closure'
20 `placement delete[] closure'
21 `managed vector constructor iterator'
22 `managed vector destructor iterator'
23 `eh vector copy constructor iterator'
24 `eh vector vbase copy constructor iterator'
25 `dynamic initializer for '
26 `dynamic atexit destructor for '
27 `vector copy constructor iterator'
28 `vector vbase copy constructor iterator'
29 `managed vector copy constructor iterator'
30 `local static thread guard'
31 Base Class Descriptor at (
32 Class Hierarchy Descriptor'
33 Complete Object Locator'
34 InitializeCriticalSectionEx
35 AppPolicyGetProcessTerminationMethod
36 !"#$%&'()*+,-./0123456789:;<=>?@abcdefghijklmnopqrstuvwxyz[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
37 !"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`ABCDEFGHIJKLMNOPQRSTUVWXYZ{|}~
38 A reboot is required following .NET installation - reboot then run installer again.
39 unzip 0.15 Copyright 1998 Gilles Vollant
40 invalid literal/length code

```

```

41 invalid stored block lengths
42 too many length or distance symbols
43 invalid bit length repeat
44 oversubscribed dynamic bit lengths tree
45 incomplete dynamic bit lengths tree
46 oversubscribed literal/length tree
47 incomplete literal/length tree
48 oversubscribed distance tree
49 empty distance tree with lengths
50 unknown compression method
51 C:\code\Squirrel.Windows\src\Setup\bin\Release\Setup.pdb
52 ConvertStringSecurityDescriptorToSecurityDescriptorW
53 InitializeCriticalSectionEx
54 SetUnhandledExceptionFilter
55 IsProcessorFeaturePresent
56 InitializeCriticalSectionAndSpinCount
57
58 abcdefghijklmnopqrstuvwxyz
59 ABCDEFGHIJKLMNOPQRSTUVWXYZ
60
61 abcdefghijklmnopqrstuvwxyz
62 ABCDEFGHIJKLMNOPQRSTUVWXYZ
63 .?AVbad_array_new_lengthstd
64 .?AV?$CCComObjectRootEx@VCCComMultiThreadModel@ATL@ATL@
65 .?AVCCComObjectRootBase@ATL@
66 .?AV?$CCComObject@VCDownloadProgressCallback@ATL@
67 .?AVCDownloadProgressCallback@
68 .?AUIBindStatusCallback@
69 .?AV?$CatlModuleT@VCCComModule@ATL@ATL@
70 Discord-1.0.9234-full.nupkgT
71 lib/net45/chrome_200_percent.pak
72 ###ddd%%&&&' 'hhh)))**++
73 lib/net45/d3dcompiler_47.dll
74 ? 0(8$4,<"2*:&6.>!1)9%5-=#3+;'7/
75 v^hEhinqn^wSwJwZw&y3y=y:yf0
76 _x@xPxHxXxDxTX(<&&<.<!<)<%,
77 <W>Q>I>Y>E>U>M>J>C>S>K>[>G>W>O>_
78 T~N~^~A~Q~I~Y~E~U~M~]~C^&
79 lib/net45/discord_wer.dll
80 KpzxHxxxjxLxDx\dpdtdXdedyD

```

```

81 Cf|qXq\qmq0q`qaqCqeqhqiqjqKqlqwqpqqqrq|
82 o}`}h}d}l}b}j}f}n}a}i}e}m}
83 IwTwLw\wBwRwJwZwFwVwNw^wAwQwIwYwEwUwMw]wCwSwKw[wGwWwOw_
84 gvshsasbscsdsesfs{sxisjskslsmsn
85 _0Z0F0V0N0^0A0Q0I0Y0E0U0MP
86 lib/net45/snapshot_blob.bin
87 lib/net45/vk_swiftshader.dll
88 lib/net45/vk_swiftshader_icd.json]
89 lib/net45/locales/en-GB.pak
90 lib/net45/locales/es-419.pak
91 lib/net45/locales/fil.pak
92 lib/net45/locales/pt-BR.pak
93 lib/net45/locales/pt-PT.pak
94 lib/net45/locales/zh-TW.pak
95 lib/net45/resources/app.asar
96 {URTT4TLT*2*,R,LRJ2*T****2T2TLL4L,****T2J,T4*LJ|
97 Discord-1.0.9234-full.nupkgPK
98 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
99 <assembly xmlns="urn:schemas-microsoft-com:asm.v1"
manifestVersion="1.0"><dependency><dependentAssembly><assemblyIdentity type="win32"
name="Microsoft.Windows.Common-Controls" version="6.0.0.0" processorArchitecture="x86"
publicKeyToken="6595b64144ccf1df" language="*"></assemblyIdentity></dependentAssembly></
dependency><trustInfo xmlns="urn:schemas-microsoft-
com:asm.v3"><security><requestedPrivileges><requestedExecutionLevel level="asInvoker"
uiAccess="false"></requestedExecutionLevel></requestedPrivileges></security></
trustInfo><compatibility xmlns="urn:schemas-microsoft-
com:compatibility.v1"><application><supportedOS Id="{e2011457-1546-43c5-a5fe-008deee3d3f0}"></
supportedOS><supportedOS Id="{35138b9a-5d96-4fbd-8e2d-a2440225f93a}"></supportedOS><supportedOS
Id="{4a2f28e3-53b9-4441-ba9c-d69d4a4a6e38}"></supportedOS><supportedOS
Id="{1f676c76-80e1-4239-95bb-83d0f6d0da78}"></supportedOS></application></compatibility></
assembly>PPADDINGXXPADDINGPADDINGXXPADDINGPADDINGXXPADDINGPADDINGXXPADDINGPADDINGXXPADDINGPADD-
INGXXPADDINGPADDINGXXPADDINGPADDINGXXPADDINGPADD

```

```

100 =0=,>0>4>8><@>D>H>L>P>^>
101 ?)??-?1?5?9?=?A?E?I?M?Q?U?Y?]?x?
102 <!<'<←<3<9<@<G<N<U<\<c<j<r<z<
103 =#)=≠5=;=A=H=O=V=]=d=k=r=z=
104 5 5(50585@5H5P5X5`5h5p5×5
105 6 6(60686@6H6P6X6`6h6p6×6
106 7 7(70787@7H7P7X7`7h7p7×7
107 8 8(80888@8H8P8X8`8h8p8×8
108 4 4(4,4044484<4@4D4L4P4T4X4\4`4d4h4t4|4
109 5 5$5(5,5054585<5@5L5P5T5X5\5`5d5h5l5p5t5×5|5
110 1$1,141<1D1L1T1\1d1l1t1|1
111 2$2,242<2D2L2T2\2d2l2t2|2
112 3$3,343<3D3L3T3\3d3l3t3|3
113 4$4,444<4D4L4T4\4d4l4t4|4
114 5$5,545<5D5L5T5\5d5l5t5|5
115 6$6,646<6D6L6T6\6d6l6t6|6
116 2 2(20282@2H2P2X2`2h2p2×2
117 3 3(30383@3H3P3X3`3h3p3×3
118 4 4(40484@4H4P4X4`4h4p4×4
119 5 5(50585@5H5P5X5`5h5p5×5
120 6 6(60686@6H6P6X6`6h6p6×6
121 7 7(70787@7H7P7X7`7h7p7×7
122 DigiCert Trusted Root G40
123 8DigiCert Trusted G4 Code Signing RSA4096 SHA384 2021 CA10
124 http://ocsp.digicert.com0A
125 5http://cacerts.digicert.com/DigiCertTrustedRootG4.crt0C
126 2http://crl3.digicert.com/DigiCertTrustedRootG4.crl0
127 8DigiCert Trusted G4 Code Signing RSA4096 SHA384 2021 CA10
128 http://www.digicert.com/CPS0
129 Mhttp://crl3.digicert.com/DigiCertTrustedG4CodeSigningRSA4096SHA3842021CA1.crl0S
130 Mhttp://crl4.digicert.com/DigiCertTrustedG4CodeSigningRSA4096SHA3842021CA1.crl0
131 http://ocsp.digicert.com0\
132 Phhttp://cacerts.digicert.com/DigiCertTrustedG4CodeSigningRSA4096SHA3842021CA1.crt0
133 8DigiCert Trusted G4 Code Signing RSA4096 SHA384 2021 CA1
134 8DigiCert Trusted G4 TimeStamping RSA4096 SHA256 2025 CA10
135 2DigiCert SHA256 RSA4096 Timestamp Responder 2025 10
136 http://ocsp.digicert.com0]
137 Qhttp://cacerts.digicert.com/DigiCertTrustedG4TimeStampingRSA4096SHA2562025CA1.crt0_
138 Nhttp://crl3.digicert.com/DigiCertTrustedG4TimeStampingRSA4096SHA2562025CA1.crl0

```

```
139 DigiCert Trusted Root G40
140 8DigiCert Trusted G4 TimeStamping RSA4096 SHA256 2025 CA10
141 http://ocsp.digicert.com0A
142 5http://cacerts.digicert.com/DigiCertTrustedRootG4.crt0C
143 2http://crl3.digicert.com/DigiCertTrustedRootG4.crl0
144 DigiCert Assured ID Root CA0
145 DigiCert Trusted Root G40
146 http://ocsp.digicert.com0C
147 7http://cacerts.digicert.com/DigiCertAssuredIDRootCA.crt0E
148 4http://crl3.digicert.com/DigiCertAssuredIDRootCA.crl0
149 8DigiCert Trusted G4 TimeStamping RSA4096 SHA256 2025 CA1
150
151 VirusTotal Data:
152 Malicious: 0
153 Suspicious: 0
154 Undetected: 69
155 Harmless: 0
156 Total Vendors: 69
157
158 Imported DLLs
159 KERNEL32.dll
160 COMCTL32.dll
161
162 Packing Analysis:
163 Likely packed or obfuscated
```

4.2.3 Output5.txt file contents, proof of successful upload to VirusTotal

```
1 Hash Values
2 MD5: d48faf20a0fabd520c9bcadd0a00b563
3 SHA256: 02cdcc5c5a84b3b52f520cff63ccda057fc3db36c8f2de81c22c3d7ef0c273cc
4
5 Extracted Strings:
6 fakefile
7 a test file
8 large strings to follow
9 0x0sbbgiahsgfabafajngajk anaj0fjwb cdbdan JAFJAOJFA, NINIFNIG
10 small strings to follow
11 tung
12 jag itns
13 sign tung
14 tung
15 pwov
16 dekdK
17 +9012fn b
18 +--ajgspnav jaogmaodsmfoax00
19 sahur
20
21 VirusTotal Data:
22 Malicious: 0
23 Suspicious: 0
24 Undetected: 61
25 Harmless: 0
26 Total Vendors: 61
27
28 Imported DLLs
29 No DLLs found or not a PE file
30 Packing Analysis:
31 Not likely packed
```

4.2.4 Ouptut6.txt contents, results of testing the verbose flag

```

1 Hash Values
2 MD5: d48faf20a0fabd520c9bcadd0a00b563
3 SHA256: 02cdcc5c5a84b3b52f520cff63ccda057fc3db36c8f2de81c22c3d7ef0c273cc
4
5 Extracted Strings:
6 fakefile
7 a test file
8 large strings to follow
9 0x0sbbgiahsqfabafajngajk anaj0fjwb cdbdan JAFJAOJFA, NINIFNIG
10 small strings to follow
11 tung
12 jag itns
13 sign tung
14 tung
15 pwov
16 dekd
17 +9012fn b
18 +---ajgspnav jaogmaodsmfoax00
19 sahur
20
21 VirusTotal Data:
22 {
23   "data": {
24     "id": "02cdcc5c5a84b3b52f520cff63ccda057fc3db36c8f2de81c22c3d7ef0c273cc",
25     "type": "file",
26     "links": {
27       "self": "https://www.virustotal.com/api/v3/files/02cdcc5c5a84b3b52f520cff63ccda057fc3db36c8f2de81c22c3d7ef0c273cc"
28     },
29     "attributes": {
30       "reputation": 0,
31       "sha1": "7b4a655f0746c0f2ca45a022efa337ce7d6cefb",
32       "sha256": "02cdcc5c5a84b3b52f520cff63ccda057fc3db36c8f2de81c22c3d7ef0c273cc",
33       "tags": [
34         "text"
35       ],
36       "times_submitted": 1,
37       "filecondis": {
38         "dhash": "be00b2b282a6aa04",
39         "raw_md5": "525e3841c61f152046613e1da0278742"
40       },

```

```

41     "last_modification_date": 1777396048,
42     "first_submission_date": 1777396030,
43     "magic": "ASCII text",
44     "last_analysis_date": 1777396030,
45     "type_description": "Text",
46     "last_analysis_stats": {
47         "malicious": 0,
48         "suspicious": 0,
49         "undetected": 61,
50         "harmless": 0,
51         "timeout": 0,
52         "confirmed-timeout": 0,
53         "failure": 0,
54         "type-unsupported": 14
55     },
56     "tlsh": "T1B4D097021A95424D2B9A351D8B4A8CE060EEB62E99F03E6AB882A1E63013C1F1930E24",
57     "size": 240,
58     "total_votes": {
59         "harmless": 0,
60         "malicious": 0
61     },
62     "type_tags": [
63         "text"
64     ],
65     "type_tag": "text",
66     "md5": "d48faf20a0fabd520c9bcadd0a00b563",
67     "last_submission_date": 1777396030,
68     "last_analysis_results": {
69         "Bkav": {
70             "method": "blacklist",
71             "engine_name": "Bkav",
72             "engine_version": "2.0.0.1",
73             "engine_update": "20260428",
74             "category": "undetected",
75             "result": null
76         },
77         "Lionic": {
78             "method": "blacklist",
79             "engine_name": "Lionic",
80             "engine_version": "8.16",

```

```

81     "engine_update": "20260428",
82     "category": "undetected",
83     "result": null
84 },
85 "MicroWorld-eScan": {
86     "method": "blacklist",
87     "engine_name": "MicroWorld-eScan",
88     "engine_version": "14.0.409.0",
89     "engine_update": "20260428",
90     "category": "undetected",
91     "result": null
92 },
93 "ClamAV": {
94     "method": "blacklist",
95     "engine_name": "ClamAV",
96     "engine_version": "1.5.2.0",
97     "engine_update": "20260428",
98     "category": "undetected",
99     "result": null
100 },
101 "CMC": {
102     "method": "blacklist",
103     "engine_name": "CMC",
104     "engine_version": "2.4.2022.1",
105     "engine_update": "20260428",
106     "category": "undetected",
107     "result": null
108 },
109 "CAT-QuickHeal": {
110     "method": "blacklist",
111     "engine_name": "CAT-QuickHeal",
112     "engine_version": "22.00",
113     "engine_update": "20260427",
114     "category": "undetected",
115     "result": null
116 },
117 "Skyhigh": {
118     "method": "blacklist",
119     "engine_name": "Skyhigh",
120     "engine_version": "v2021.2.0+4045",

```

```

121     "engine_update": "20260428",
122     "category": "undetected",
123     "result": null
124 },
125 "ALYac": {
126     "method": "blacklist",
127     "engine_name": "ALYac",
128     "engine_version": "2.0.0.10",
129     "engine_update": "20260428",
130     "category": "undetected",
131     "result": null
132 },
133 "Malwarebytes": {
134     "method": "blacklist",
135     "engine_name": "Malwarebytes",
136     "engine_version": "3.1.0.231",
137     "engine_update": "20260428",
138     "category": "undetected",
139     "result": null
140 },
141 "VIPRE": {
142     "method": "blacklist",
143     "engine_name": "VIPRE",
144     "engine_version": "6.0.0.35",
145     "engine_update": "20260428",
146     "category": "undetected",
147     "result": null
148 },
149 "Sangfor": {
150     "method": "blacklist",
151     "engine_name": "Sangfor",
152     "engine_version": "2.22.3.0",
153     "engine_update": "20260427",
154     "category": "undetected",
155     "result": null
156 },
157 "K7AntiVirus": {
158     "method": "blacklist",
159     "engine_name": "K7AntiVirus",
160     "engine_version": "14.49.59339",|

```

```

161     "engine_update": "20260428",
162     "category": "undetected",
163     "result": null
164 },
165 "K7GW": {
166     "method": "blacklist",
167     "engine_name": "K7GW",
168     "engine_version": "14.49.59340",
169     "engine_update": "20260428",
170     "category": "undetected",
171     "result": null
172 },
173 "CrowdStrike": {
174     "method": "blacklist",
175     "engine_name": "CrowdStrike",
176     "engine_version": "1.0",
177     "engine_update": "20230417",
178     "category": "undetected",
179     "result": null
180 },
181 "Arcabit": {
182     "method": "blacklist",
183     "engine_name": "Arcabit",
184     "engine_version": "2025.0.0.23",
185     "engine_update": "20260428",
186     "category": "undetected",
187     "result": null
188 },
189 "VirIT": {
190     "method": "blacklist",
191     "engine_name": "VirIT",
192     "engine_version": "9.5.1196",
193     "engine_update": "20260428",
194     "category": "undetected",
195     "result": null
196 },
197 "Symantec": {
198     "method": "blacklist",
199     "engine_name": "Symantec",
200     "engine_version": "1.22.0.0",|

```

```
201     "engine_update": "20260428",
202     "category": "undetected",
203     "result": null
204 },
205 "ESET-NOD32": {
206     "method": "blacklist",
207     "engine_name": "ESET-NOD32",
208     "engine_version": "18.2.18.0",
209     "engine_update": "20260428",
210     "category": "undetected",
211     "result": null
212 },
213 "TrendMicro-HouseCall": {
214     "method": "blacklist",
215     "engine_name": "TrendMicro-HouseCall",
216     "engine_version": "24.550.0.1002",
217     "engine_update": "20260428",
218     "category": "undetected",
219     "result": null
220 },
221 "Avast": {
222     "method": "blacklist",
223     "engine_name": "Avast",
224     "engine_version": "23.9.8494.0",
225     "engine_update": "20260322",
226     "category": "undetected",
227     "result": null
228 },
229 "Cynet": {
230     "method": "blacklist",
231     "engine_name": "Cynet",
232     "engine_version": "4.0.3.4",
233     "engine_update": "20260428",
234     "category": "undetected",
235     "result": null
236 },
237 "Kaspersky": {
238     "method": "blacklist",
239     "engine_name": "Kaspersky",
240     "engine_version": "22.0.1.28",
```

```
241     "engine_update": "20260428",
242     "category": "undetected",
243     "result": null
244 },
245 "BitDefender": {
246     "method": "blacklist",
247     "engine_name": "BitDefender",
248     "engine_version": "7.2",
249     "engine_update": "20260428",
250     "category": "undetected",
251     "result": null
252 },
253 "NANO-Antivirus": {
254     "method": "blacklist",
255     "engine_name": "NANO-Antivirus",
256     "engine_version": "1.0.170.26895",
257     "engine_update": "20260428",
258     "category": "undetected",
259     "result": null
260 },
261 "ViRobot": {
262     "method": "blacklist",
263     "engine_name": "ViRobot",
264     "engine_version": "2014.3.20.0",
265     "engine_update": "20260428",
266     "category": "undetected",
267     "result": null
268 },
269 "Rising": {
270     "method": "blacklist",
271     "engine_name": "Rising",
272     "engine_version": "25.0.0.28",
273     "engine_update": "20260428",
274     "category": "undetected",
275     "result": null
276 },
277 "Sophos": {
278     "method": "blacklist",
279     "engine_name": "Sophos",
280     "engine_version": "3.4.1|.0",
```

```
281     "engine_update": "20260428",
282     "category": "undetected",
283     "result": null
284 },
285 "F-Secure": {
286     "method": "blacklist",
287     "engine_name": "F-Secure",
288     "engine_version": "18.10.1547.307",
289     "engine_update": "20260428",
290     "category": "undetected",
291     "result": null
292 },
293 "DrWeb": {
294     "method": "blacklist",
295     "engine_name": "DrWeb",
296     "engine_version": "7.0.75.2070",
297     "engine_update": "20260428",
298     "category": "undetected",
299     "result": null
300 },
301 "Zillya": {
302     "method": "blacklist",
303     "engine_name": "Zillya",
304     "engine_version": "2.0.0.5589",
305     "engine_update": "20260427",
306     "category": "undetected",
307     "result": null
308 },
309 "TrendMicro": {
310     "method": "blacklist",
311     "engine_name": "TrendMicro",
312     "engine_version": "24.550.0.1002",
313     "engine_update": "20260428",
314     "category": "undetected",
315     "result": null
316 },
317 "McAfeeD": {
318     "method": "blacklist",
319     "engine_name": "McAfeeD",
320     "engine_version": "1.2.0.14532",
```

```
321     "engine_update": "20260428",
322     "category": "undetected",
323     "result": null
324 },
325 "CTX": {
326     "method": "blacklist",
327     "engine_name": "CTX",
328     "engine_version": "2024.8.29.1",
329     "engine_update": "20260428",
330     "category": "undetected",
331     "result": null
332 },
333 "Emsisoft": {
334     "method": "blacklist",
335     "engine_name": "Emsisoft",
336     "engine_version": "2024.8.0.61147",
337     "engine_update": "20260428",
338     "category": "undetected",
339     "result": null
340 },
341 "Ikarus": {
342     "method": "blacklist",
343     "engine_name": "Ikarus",
344     "engine_version": "6.4.16.0",
345     "engine_update": "20260428",
346     "category": "undetected",
347     "result": null
348 },
349 "Jiangmin": {
350     "method": "blacklist",
351     "engine_name": "Jiangmin",
352     "engine_version": "16.0.100",
353     "engine_update": "20260427",
354     "category": "undetected",
355     "result": null
356 },
357 "Varist": {
358     "method": "blacklist",
359     "engine_name": "Varist",
360     "engine_version": "6.6.1.3",
```

```
361     "engine_update": "20260428",
362     "category": "undetected",
363     "result": null
364 },
365 "Avira": {
366     "method": "blacklist",
367     "engine_name": "Avira",
368     "engine_version": "8.3.3.24",
369     "engine_update": "20260428",
370     "category": "undetected",
371     "result": null
372 },
373 "Antiy-AVL": {
374     "method": "blacklist",
375     "engine_name": "Antiy-AVL",
376     "engine_version": "3.0",
377     "engine_update": "20260428",
378     "category": "undetected",
379     "result": null
380 },
381 "Kingsoft": {
382     "method": "blacklist",
383     "engine_name": "Kingsoft",
384     "engine_version": "None",
385     "engine_update": "20260428",
386     "category": "undetected",
387     "result": null
388 },
389 "Gridinsoft": {
390     "method": "blacklist",
391     "engine_name": "Gridinsoft",
392     "engine_version": "1.0.244.174",
393     "engine_update": "20260428",
394     "category": "undetected",
395     "result": null
396 },
397 "Xcitium": {
398     "method": "blacklist",
399     "engine_name": "Xcitium",
400     "engine_version": "38603",
```

```
401     "engine_update": "20260428",
402     "category": "undetected",
403     "result": null
404 },
405 "Microsoft": {
406     "method": "blacklist",
407     "engine_name": "Microsoft",
408     "engine_version": "1.1.26030.3008",
409     "engine_update": "20260428",
410     "category": "undetected",
411     "result": null
412 },
413 "SUPERAntiSpyware": {
414     "method": "blacklist",
415     "engine_name": "SUPERAntiSpyware",
416     "engine_version": "5.6.0.1032",
417     "engine_update": "20260428",
418     "category": "undetected",
419     "result": null
420 },
421 "ZoneAlarm": {
422     "method": "blacklist",
423     "engine_name": "ZoneAlarm",
424     "engine_version": "6.24-114820531",
425     "engine_update": "20260428",
426     "category": "undetected",
427     "result": null
428 },
429 "GData": {
430     "method": "blacklist",
431     "engine_name": "GData",
432     "engine_version": "GD:27.44353AVA:64.31132",
433     "engine_update": "20260428",
434     "category": "undetected",
435     "result": null
436 },
437 "Google": {
438     "method": "blacklist",
439     "engine_name": "Google",
440     "engine_version": "1777392067",|
```

```
441     "engine_update": "20260428",
442     "category": "undetected",
443     "result": null
444 },
445 "AhnLab-V3": {
446     "method": "blacklist",
447     "engine_name": "AhnLab-V3",
448     "engine_version": "3.30.0.10666",
449     "engine_update": "20260428",
450     "category": "undetected",
451     "result": null
452 },
453 "Acronis": {
454     "method": "blacklist",
455     "engine_name": "Acronis",
456     "engine_version": "1.2.0.121",
457     "engine_update": "20240328",
458     "category": "undetected",
459     "result": null
460 },
461 "VBA32": {
462     "method": "blacklist",
463     "engine_name": "VBA32",
464     "engine_version": "5.6.0",
465     "engine_update": "20260428",
466     "category": "undetected",
467     "result": null
468 },
469 "TACHYON": {
470     "method": "blacklist",
471     "engine_name": "TACHYON",
472     "engine_version": "2026-04-23.02",
473     "engine_update": "20260423",
474     "category": "undetected",
475     "result": null
476 },
477 "Zoner": {
478     "method": "blacklist",
479     "engine_name": "Zoner",
480     "engine_version": "2.2.2.0",|
```

```
481     "engine_update": "20260428",
482     "category": "undetected",
483     "result": null
484 },
485 "Tencent": {
486     "method": "blacklist",
487     "engine_name": "Tencent",
488     "engine_version": "1.0.0.1",
489     "engine_update": "20260428",
490     "category": "undetected",
491     "result": null
492 },
493 "Yandex": {
494     "method": "blacklist",
495     "engine_name": "Yandex",
496     "engine_version": "5.5.2.24",
497     "engine_update": "20260428",
498     "category": "undetected",
499     "result": null
500 },
501 "TrellixENS": {
502     "method": "blacklist",
503     "engine_name": "TrellixENS",
504     "engine_version": "6.0.6.653",
505     "engine_update": "20260428",
506     "category": "undetected",
507     "result": null
508 },
509 "huorong": {
510     "method": "blacklist",
511     "engine_name": "huorong",
512     "engine_version": "fccbab7:fccbab7:8a0870e:8a0870e",
513     "engine_update": "20260428",
514     "category": "undetected",
515     "result": null
516 },
517 "MaxSecure": {
518     "method": "blacklist",
519     "engine_name": "MaxSecure",
520     "engine_version": "1.0.0.1",
```

```

521     "engine_update": "20260428",|
522     "category": "undetected",
523     "result": null
524 },
525 "Fortinet": {
526     "method": "blacklist",
527     "engine_name": "Fortinet",
528     "engine_version": "7.0.30.0",
529     "engine_update": "20260428",
530     "category": "undetected",
531     "result": null
532 },
533 "AVG": {
534     "method": "blacklist",
535     "engine_name": "AVG",
536     "engine_version": "23.9.8494.0",
537     "engine_update": "20260322",
538     "category": "undetected",
539     "result": null
540 },
541 "Panda": {
542     "method": "blacklist",
543     "engine_name": "Panda",
544     "engine_version": "4.6.4.2",
545     "engine_update": "20260428",
546     "category": "undetected",
547     "result": null
548 },
549 "alibabacloud": {
550     "method": "blacklist",
551     "engine_name": "alibabacloud",
552     "engine_version": "2.2.0",
553     "engine_update": "20250321",
554     "category": "undetected",
555     "result": null
556 },
557 "Avast-Mobile": {
558     "method": "blacklist",
559     "engine_name": "Avast-Mobile",
560     "engine_version": "260428-00",

```

```

561     "engine_update": "20260428",
562     "category": "type-unsupported",
563     "result": null
564 },
565 "SymantecMobileInsight": {
566     "method": "blacklist",
567     "engine_name": "SymantecMobileInsight",
568     "engine_version": "2.0",
569     "engine_update": "20260123",
570     "category": "type-unsupported",
571     "result": null
572 },
573 "BitDefenderFalx": {
574     "method": "blacklist",
575     "engine_name": "BitDefenderFalx",
576     "engine_version": "2.0.936",
577     "engine_update": "20251216",
578     "category": "type-unsupported",
579     "result": null
580 },
581 "DeepInstinct": {
582     "method": "blacklist",
583     "engine_name": "DeepInstinct",
584     "engine_version": "5.0.0.8",
585     "engine_update": "20260428",
586     "category": "type-unsupported",
587     "result": null
588 },
589 "Elastic": {
590     "method": "blacklist",
591     "engine_name": "Elastic",
592     "engine_version": "4.0.258",
593     "engine_update": "20260428",
594     "category": "type-unsupported",
595     "result": null
596 },
597 "Webroot": {
598     "method": "blacklist",
599     "engine_name": "Webroot",
600     "engine_version": "1.9.0.8",

```

```

601     "engine_update": "20250227",
602     "category": "type-unsupported",
603     "result": null
604 },
605 "APEX": {
606     "method": "blacklist",
607     "engine_name": "APEX",
608     "engine_version": "6.774",
609     "engine_update": "20260428",
610     "category": "type-unsupported",
611     "result": null
612 },
613 "Paloalto": {
614     "method": "blacklist",
615     "engine_name": "Paloalto",
616     "engine_version": "0.9.0.1003",
617     "engine_update": "20260428",
618     "category": "type-unsupported",
619     "result": null
620 },
621 "Alibaba": {
622     "method": "blacklist",
623     "engine_name": "Alibaba",
624     "engine_version": "0.3.0.5",
625     "engine_update": "20190527",
626     "category": "type-unsupported",
627     "result": null
628 },
629 "Trapmine": {
630     "method": "blacklist",
631     "engine_name": "Trapmine",
632     "engine_version": "4.0.11.0",
633     "engine_update": "20260331",
634     "category": "type-unsupported",
635     "result": null
636 },
637 "Cylance": {
638     "method": "blacklist",
639     "engine_name": "Cylance",
640     "engine_version": "3.0.0.0",

```

```

641     "engine_update": "20260423",
642     "category": "type-unsupported",
643     "result": null
644 },
645 "SentinelOne": {
646     "method": "blacklist",
647     "engine_name": "SentinelOne",
648     "engine_version": "7.6.2.19",
649     "engine_update": "20260324",
650     "category": "type-unsupported",
651     "result": null
652 },
653 "tehttris": {
654     "method": "blacklist",
655     "engine_name": "tehttris",
656     "engine_version": null,
657     "engine_update": "20260428",
658     "category": "type-unsupported",
659     "result": null
660 },
661 "Trustlook": {
662     "method": "blacklist",
663     "engine_name": "Trustlook",
664     "engine_version": "1.0",
665     "engine_update": "20260428",
666     "category": "type-unsupported",
667     "result": null
668 }
669 },
670 "ssdeep": "6:BKErEmoXDXPH9YQ8TM5X9hmGRmgvZvwqCNxr0HHPEmWCZsn:BKnTXuM5zN/jAZ08mDsn",
671 "meaningful_name": "fakefile.txt",
672 "magika": "TXT",
673 "trid": [
674     {
675         "file_type": "file seems to be plain text/ASCII",
676         "probability": 0.0
677     }
678 ],
679 "names": [
680     "fakefile.txt"

```

```

681     ],
682     "type_extension": "txt",
683     "javascript_info": {
684         "tags": [
685             "malformed"
686         ]
687     },
688     "unique_sources": 1
689 }
690 }
691 }Malicious: 0
692 Suspicious: 0
693 Undetected: 61
694 Harmless: 0
695 Total Vendors: 61
696
697 Imported DLLs
698 No DLLs found or not a PE file
699 Packing Analysis:
700 Not likely packed

```

4.2.5 Output2.txt contents, results of testing the docker implementation of the script

```

1 Hash Values
2 MD5: 44d88612fea8a8f36de82e1278abb02f
3 SHA256: 275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f
4
5 Extracted Strings:
6 X50!P%QAP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
7
8 VirusTotal Data:
9 Malicious: 66
10 Suspicious: 0
11 Undetected: 2
12 Harmless: 0
13 Total Vendors: 68
14
15 Imported DLLs
16 No DLLs found or not a PE file
17 Packing Analysis:
18 Not likely packed

```

APPENDIX B – SCRIPT CODE

```
import os

import sys

import requests

import hashlib

import json

import string

import argparse

import pefile

import math

import mimetypes

from colorama import Fore, Back, Style


def hash_file(filepath):

    """

    Computes the MD5 and SHA256 hash of the chosen file

    Args:

        filepath -- Path to the file to be hashed

    Returns:

        md5.hexdigest() -- MD5 hash value as hexadecimal string

        sha256.hexdigest() -- SHA256 hash value as hexadecimal string

    """

    BUF_SIZE = 65536 # Set the size of bytes a file will be read with for hashing

    md5 = hashlib.md5()

    sha256 = hashlib.sha256()

    try:

        with open(filepath, 'rb') as f:

            while True:

                data = f.read(BUF_SIZE)

                if not data:
```

```

        break
        md5.update(data)
        sha256.update(data)
except IOError as e:
    print(f"Error reading file {filepath}: {e}")
    sys.exit()

print(f"MD5: {md5.hexdigest()}")
print(f"SHA256: {sha256.hexdigest()}")

return md5.hexdigest(), sha256.hexdigest()

def query_virustotal(sha256_hash, api_key):
    """
    Queries the VirusTotal API to receive information on the file

    Args:
        sha256_hash -- SHA256 hash of the file
        api_key -- VirusTotal API key for authentication

    Returns:
        response -- A tuple containing API response data and object
    """

    url = f"https://www.virustotal.com/api/v3/files/{sha256_hash}"

    headers = {
        "accept": "application/json",
        "x-apikey": api_key
    }

    try:
        response = requests.get(url, headers=headers)
        response.raise_for_status()
        return response.json(), response
    except Exception as e:
        print(f"Error querying VirusTotal: {e}")
        sys.exit()

def string_extraction(filepath, min_length):
    """

```

Extracts and returns printable ASCII strings of specified minimum length

Args:

 filepath -- Path to the file to be analysed
 min_length -- Minimum length for the extracted strings

Returns:

 extracted_strings -- A list of extracted strings
"""

extracted_strings = []

try:

 with open(filepath, 'rb') as f:
 data = f.read()

except IOError as e:

 print(f"Error reading file {filepath}: {e}")
 sys.exit()

current = ""

Reads data in byte chunks and stores string using it

for byte in data:

 char = chr(byte)

 if char in string.printable and char not in '\n\r\t\x0b\x0c':
 current+= char

 else:

 if len(current) >= min_length:
 extracted_strings.append(current)
 current = ""

if len(current) >= min_length:

 extracted_strings.append(current)

return extracted_strings

def imported_dll(filepath):

"""

Extracts and returns a list of the DLL's imported by the file

Args:

 filepath -- Path to the file to be analysed

Returns:

 dlls -- a list of DLLs imported by the file
"""

dlls = []

```

try:
    pe = pefile.PE(filepath)

    # Checks for import attribute and decodes name of imported DLL
    if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):
        for imp in pe.DIRECTORY_ENTRY_IMPORT:
            dllname = imp.dll.decode('utf-8', errors='ignore')
            dlls.append(dllname)

except Exception as e:
    print("PE Parsing error:", e)

return dlls


def packing_check(filepath):
    """
    Estimates if a file is packed using entropy and section name analysis

    Args:
        filepath -- Path to the file to be analysed

    Returns:
        Packed -- True if the file is likely packed, false otherwise
    """
    packed = False
    susNames = [".upx", ".vmp0", ".MPRESS", ".aspack", "adata", ".themida"]

    try:
        pe = pefile.PE(filepath)

        # Iterates through sections of PE file, gets name and data from them
        for section in pe.sections:
            name = section.Name.decode(errors="ignore").strip("\x00")
            data = section.get_data()

            entropyValue = calculate_entropy(data)

            if entropyValue > 7.0:
                packed = True

            for sus in susNames:
                if sus.lower() in name.lower():
                    packed = True
    
```

```

except Exception as e:
    print("Packing check error:", e)

return packed


def calculate_entropy(data):
    """
    calculates entropy using Shannon's Formula and returns results

    Args:
        data -- Data to analyse

    Returns:
        entropy -- the entropy value
    """

    if not data:
        return 0

    entropy = 0
    for x in range(256):
        p_x = data.count(bytes([x])) / len(data)
        if p_x > 0:
            entropy += -p_x * math.log2(p_x)

    return entropy


def upload_file(filepath, api_key):
    """
    uses VirusTotal API to upload file to virustotal for analysis

    Args:
        filepath -- Path to file to be uploaded
        api_key -- VirusTotal API key for authentication
    """

    url = "https://www.virustotal.com/api/v3/files"

    filetype = mimetypes.guess_type(filepath)[0]
    files = {"file": (filepath, open(filepath, "rb"), filetype)}

    headers = {

```

```

    "accept": "application/json",
    "x-apikey": api_key,
}

try:
    response = requests.post(url, files=files, headers=headers)
    print("File upload successful")
except Exception as e:
    print(f"Error uplaoding file: {e}")

sys.exit()

def write_to_file( outputfile, md5, sha256, strings, args, data, numMal,
numSus, numUnd, numHless, totalCheck, dlls, packed ):

    """ writes analysis results to an output file

Args:
    outputfile -- Path to the output file
    md5 -- MD5 hash of the file
    sha256 -- SHA256 hash of the file
    strings -- Extracted strings from the file
    args -- command-line arguments
    data -- Data parsed from VirusTotal API response
    numMal -- Number of malicious detections
    numSus -- Number of suspicious detections
    numUnd -- Number of undetected detections
    numHless -- Number of harmless detections
    totalCheck -- Total number of vendors that processed the file
    dlls -- List of imported DLLs
    packed -- estimate of whether the file is packed or not
    """
    try:
        with open(outputfile, 'w') as f:
            f.write("Hash Values\n")
            f.write("MD5: {0}\n".format(md5))
            f.write("SHA256: {0}\n".format(sha256))

            f.write("\nExtracted Strings:\n")
            for s in strings:
                f.write(s + "\n")

            f.write("\nVirusTotal Data:\n")
            if args.verbose:
                f.write(json.dumps(data, indent=2))

            f.write(f"Malicious: {numMal}\n")
            f.write(f"Suspicious: {numSus}\n")
            f.write(f"Undetected: {numUnd}\n")

```

```

        f.write(f"Harmless: {numHless}\n")
        f.write(f"Total Vendors: {totalCheck}\n")

        f.write("\nImported DLLs\n")
        if dlls:
            for d in dlls:
                f.write(d + "\n")
        else:
            f.write("No DLLs found or not a PE file")

        f.write("\nPacking Analysis:\n")

        if packed == True:
            f.write("Likely packed or obfuscated")
        else:
            f.write("Not likely packed")
except IOError as e:
    print(f"Error writing to file {outputfile}: {e}")
    sys.exit()

def main():

    """ Main function to perform static analysis on a given file

    Takes arguments from the command line, parses json data
    and calls other functions to compute hashes, query VirusTotal,
    extract strings, check imported DLLs perform packing analysis
    and write results to an output file
    """

    # Takes arguments from the command line and parses them
    parser = argparse.ArgumentParser(
        description="Simple Malware Analysis tool. " \
        "Usage: python Staticfy.py [filepath] [outputfile]"
    )
    parser.add_argument(
        "filepath", help="Define a file for the tool to analyse"
    )
    parser.add_argument("outputfile",
        help="Define a file for the results of the analysis"
    )
    parser.add_argument("-m", "--min-length", type=int, default=4,
        help="Minimum string length for extraction (default: 4)"
    )
    parser.add_argument(
        "-v", "--verbose",
        help="Write all data from VirusTotal to file",
        action="store_true"
    )

```

```

parser.add_argument(
    "-u", "--upload", help="Upload a file to VirusTotal",
    action="store_true"
)

args = parser.parse_args()

filepath = args.filepath
outputfile = args.outputfile
min_length = args.min_length
if min_length < 1:
    print("Minimum length must be 1 or greater")
    sys.exit()

# Use environment variables to get VirusTotal API key
api_key = os.environ.get("VT_API_KEY")

md5, sha256 = hash_file(filepath)

if args.upload:
    upload_file(filepath, api_key)

data, response = query_virustotal(sha256, api_key)

# Parse json data and store in variables
stats = data["data"]["attributes"]["last_analysis_stats"]
numMal = (stats['malicious'])
numSus = (stats['suspicious'])
numUnd = (stats['undetected'])
numHless = (stats['harmless'])
totalCheck = numSus + numMal + numUnd + numHless

strings = string_extraction(filepath, min_length)
dlls = imported_dll(filepath)
packed = packing_check(filepath)

# Print nature of file and results
if numMal > (totalCheck * 0.5):
    print(f"Likely {Fore.RED} [*]Malicious[*] {Style.RESET_ALL}")
    print(f"This file was identified as malicious by {numMal} of "
          f"{totalCheck} Security Vendors")
elif numMal == 0 and numSus == 0:
    print(f"Likely {Fore.GREEN} [✓]SAFE[✓] {Style.RESET_ALL}")
    print(f"0 Security Vendors identified this file as malicious or suspicious")
else:
    print(f"{Fore.CYAN} [?]Unknown[?] {Style.RESET_ALL}")
    print(f"This file was identified as malicious {numMal} times, "
          f"suspicious {numSus} times, harmless {numHless} times "
          f"and was undetected {numUnd} times out of {totalCheck} ")

```

```
        "security vendors who processed it")

write_to_file(
    outputfile, md5, sha256, strings, args, data, numMal, numSus, numUnd,
    numHless, totalCheck,dlls, packed
)

if name == "main":

    main()
```

APPENDIX C – CONTENTS OF THE DOCKERFILE

```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 RUN pip install requests
6 RUN pip install pefile
7 RUN pip install colorama
8
9 COPY Staticfy.py /app/Staticfy.py
10 |
11 CMD ["python", "Staticfy.py"]
```

APPENDIX D – README FILE CONTENTS

Staticfy - Automated Static Malware Analysis Tool

Staticfy is a lightweight python based script that allows for the automation of some areas of static analysis on a file. It uses VirusTotal to determine the nature of the file, performs string extraction, packing detection, DLL extraction and writes the results to a file for further analysis. The script has been adapted to docker use for portability and ease of use.

To run ensure python and Docker are installed on your system.

Ensure you have a VirusTotal API key, this can be obtained by signing up to the VirusTotal website

Setup:-

Ensure you have the following files in a folder:

- Staticfy.py
- the file you would like to analyse
- Dockerfile

Building the docker image:-

Use the following command to build the docker image in the directory where dockerfile is located:

```
docker build -t staticfy-malware-analysis .
```

Running the tool:-

To run the tool, use docker to open an interactive shell within the container. The command is shown below:

```
docker run --rm -it -v $(pwd):/app -e VT_API_KEY="myapikey" staticfy-malware-analysis /bin/bash
```

Then the tool can be run as standard using the command usage shown below:

```
python Staticfy.py [target file] [output file] [options]
```

Options:-

- m [number] or --min_length [number]: Minimum string length for extraction, default is 4.
- v or --verbose: Enables verbose output from the VirusTotal API, which is written to the output file
- u or --upload: Uploads the file to VirusTotal for analysis